

Application Layer

Lecture#07-12



Dr. Sanjeev Patel

Asst Professor, CSE Dept.

NIT Rourkela

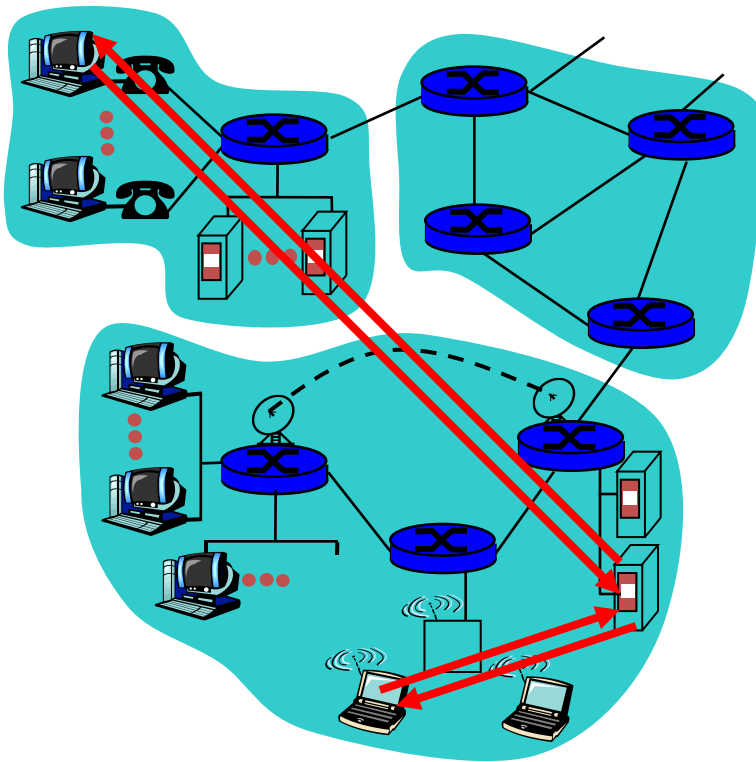
Outline

- Principles of network applications
- Client Server Model
- Web and HTTP
- FTP
- Electronic Mail
- SMTP
- DNS

Application architectures

- Client-server
- Peer-to-peer (P2P)
- Hybrid of client-server and P2P

Client-server architecture [6]



server:

- always-on host
- permanent IP address
- server farms for scaling

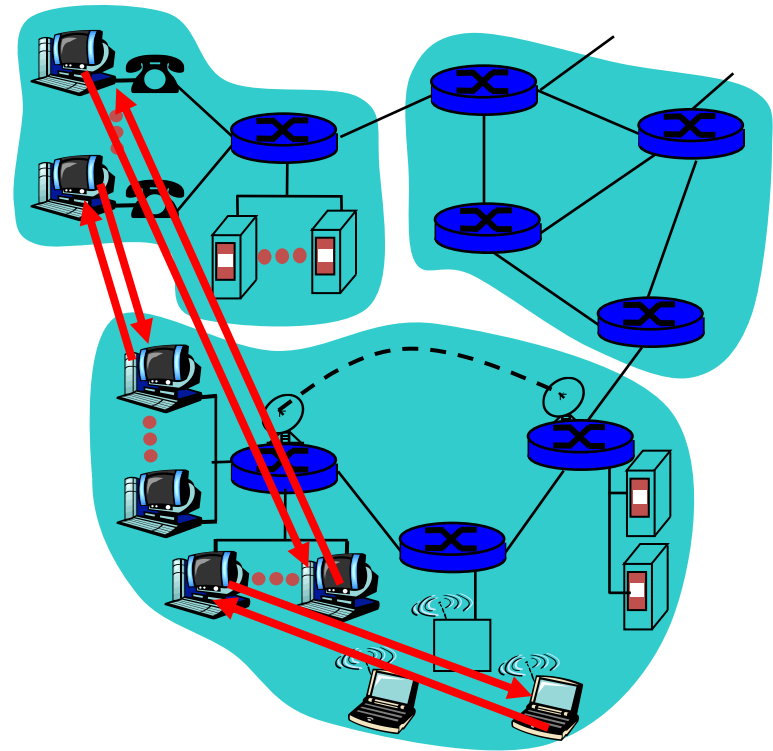
clients:

- communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- do not communicate directly with each other

Pure P2P architecture [6]

- no always-on server
- arbitrary end systems directly communicate
- peers are intermittently connected and change IP addresses
- example: Gnutella

Highly scalable but difficult to manage



Hybrid of client-server and P2P

Skype

- Internet telephony app
- Finding address of remote party: centralized server(s)
- Client-client connection is direct (not through server)

Instant messaging

- Chatting between two users is P2P
- Presence detection/location centralized:
 - User registers its IP address with central server when it comes online
 - User contacts central server to find IP addresses of buddies

Processes communicating

Process: program running within a host.

- within same host, two processes communicate using **inter-process communication** (defined by OS).
- processes in different hosts communicate by exchanging **messages**

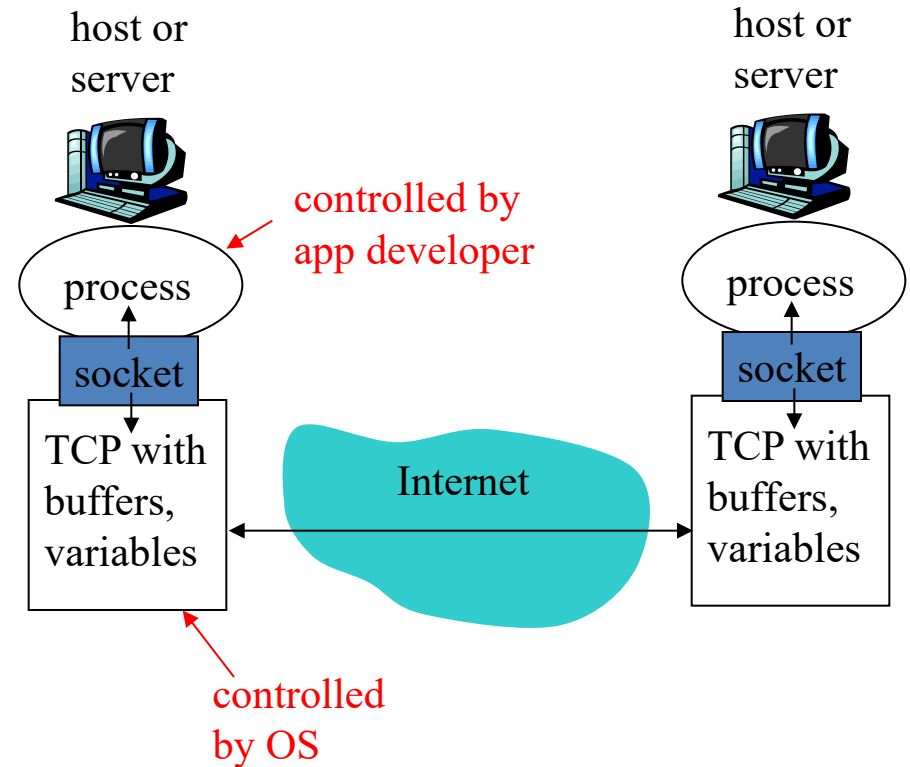
Client process: process that initiates communication

Server process: process that waits to be contacted

- **Note:** applications with P2P architectures have client processes & server processes

Sockets [6]

- process sends/receives messages to/from its **socket**
- socket analogous to door
 - sending process shoves(push) message out door
 - sending process relies on transport infrastructure on other side of door which brings message to socket at receiving process



- ❑ API: (1) choice of transport protocol; (2) ability to fix a few parameters (**lots more on this later**)

Addressing processes

- to receive messages, process must have *identifier*
- host device has unique 32-bit IP address
- **Q:** does IP address of host on which process runs suffice for identifying the process?

Addressing processes

- to receive messages, process must have *identifier*
- host device has unique 32-bit IP address
- **Q:** does IP address of host on which process runs suffice for identifying the process?
 - **Answer:** NO, many processes can be running on same host
- *identifier* includes both **IP address** and **port numbers** associated with process on host.
- Example port numbers:
 - HTTP server: 80
 - Mail server: 25
- **to send HTTP message to gaia.cs.umass.edu web server:**
 - **IP address:** 128.119.245.12
 - **Port number:** 80
- **more shortly...**

App-layer protocol defines

- Types of messages exchanged,
 - e.g., request, response
- Message syntax:
 - what fields in messages & how fields are delineated
- Message semantics
 - meaning of information in fields
- Rules for when and how processes send & respond to messages

Public-domain protocols:

- defined in RFCs
- allows for interoperability
- e.g., HTTP, SMTP

Proprietary protocols:

- e.g., KaZaA

What transport service does an app need? [6]

Data loss

- some apps (e.g., audio) can tolerate some loss
- other apps (e.g., file transfer, telnet) require 100% reliable data transfer

Timing

- some apps (e.g., Internet telephony, interactive games) require low delay to be “effective”

Bandwidth

- ❑ some apps (e.g., multimedia) require minimum amount of bandwidth to be “effective”
- ❑ other apps (“elastic apps”) make use of whatever bandwidth they get

Transport service requirements of common apps [6]

Application	Data loss	Bandwidth	Time Sensitive
file transfer	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5kbps-1Mbps video:10kbps-5Mbps	yes, 100's msec
stored audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	few kbps up	yes, 100's msec
instant messaging	no loss	elastic	yes and no

Internet transport protocols services [6]

TCP service:

- *connection-oriented*: setup required between client and server processes
- *reliable transport* between sending and receiving process
- *flow control*: sender won't overwhelm receiver
- *congestion control*: throttle sender when network overloaded
- *does not provide*: timing, minimum bandwidth guarantees

UDP service:

- unreliable data transfer between sending and receiving process
- does not provide: connection setup, reliability, flow control, congestion control, timing, or bandwidth guarantee

Q: why bother? Why is there a UDP?

Internet apps: application, transport protocols [6]

Application		Application layer protocol	Underlying transport protocol
remote	e-mail	SMTP [RFC 2821]	TCP
	terminal access	Telnet [RFC 854]	TCP
	Web	HTTP [RFC 2616]	TCP
	file transfer	FTP [RFC 959]	TCP
	streaming multimedia	proprietary (e.g. RealNetworks)	TCP or UDP
	Internet telephony	proprietary (e.g., Vonage,Dialpad)	typically UDP

Web and HTTP [6]

First some jargon

- **Web page** consists of **objects**
- Object can be HTML file, JPEG image, Java applet, audio file,...
- Web page consists of **base HTML-file** which includes several referenced objects
- Each object is addressable by a **URL**
- Example URL:

`www.someschool.edu/someDept/pic.gif`

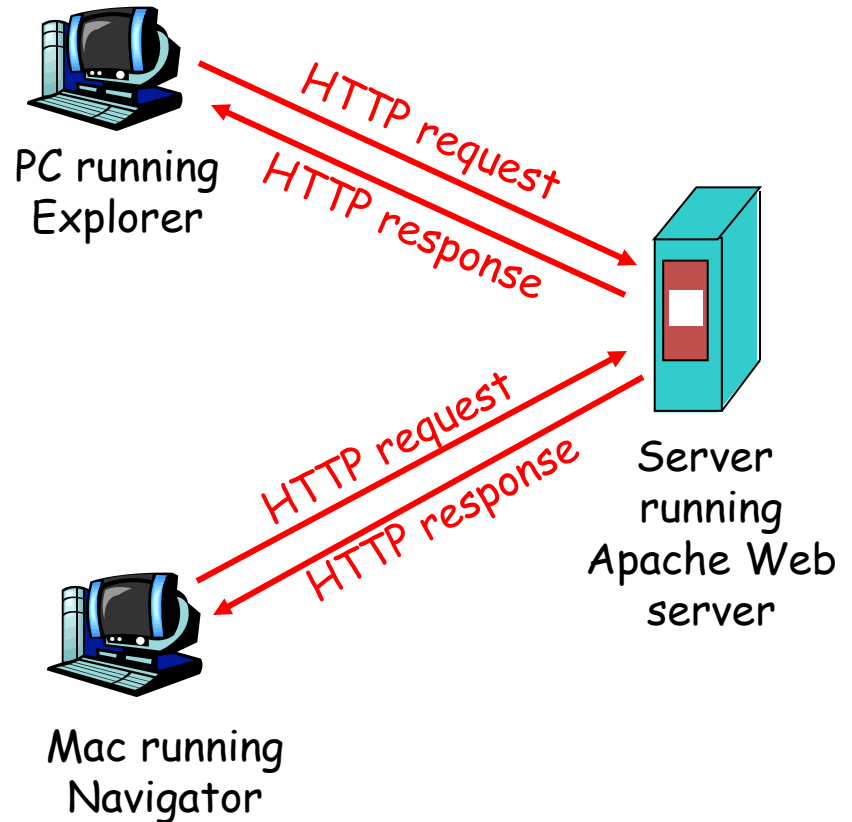
host name

path name

HTTP overview [6]

HTTP: hypertext transfer protocol

- Web's application layer protocol
- client/server model
 - *client*: browser that requests, receives, “displays” Web objects
 - *server*: Web server sends objects in response to requests
- HTTP 1.0: RFC 1945
- HTTP 1.1: RFC 2068



HTTP overview (continued) [6]

Uses TCP:

- client initiates TCP connection (creates socket) to server, port 80
- server accepts TCP connection from client
- HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

HTTP is “stateless”

- server maintains no information about past client requests

Protocols that maintain “state” are complex! aside

- ☐ past history (state) must be maintained
- ☐ if server/client crashes, their views of “state” may be inconsistent, must be reconciled

HTTP connections

Nonpersistent HTTP

- At most one object is sent over a TCP connection.
- HTTP/1.0 uses nonpersistent HTTP

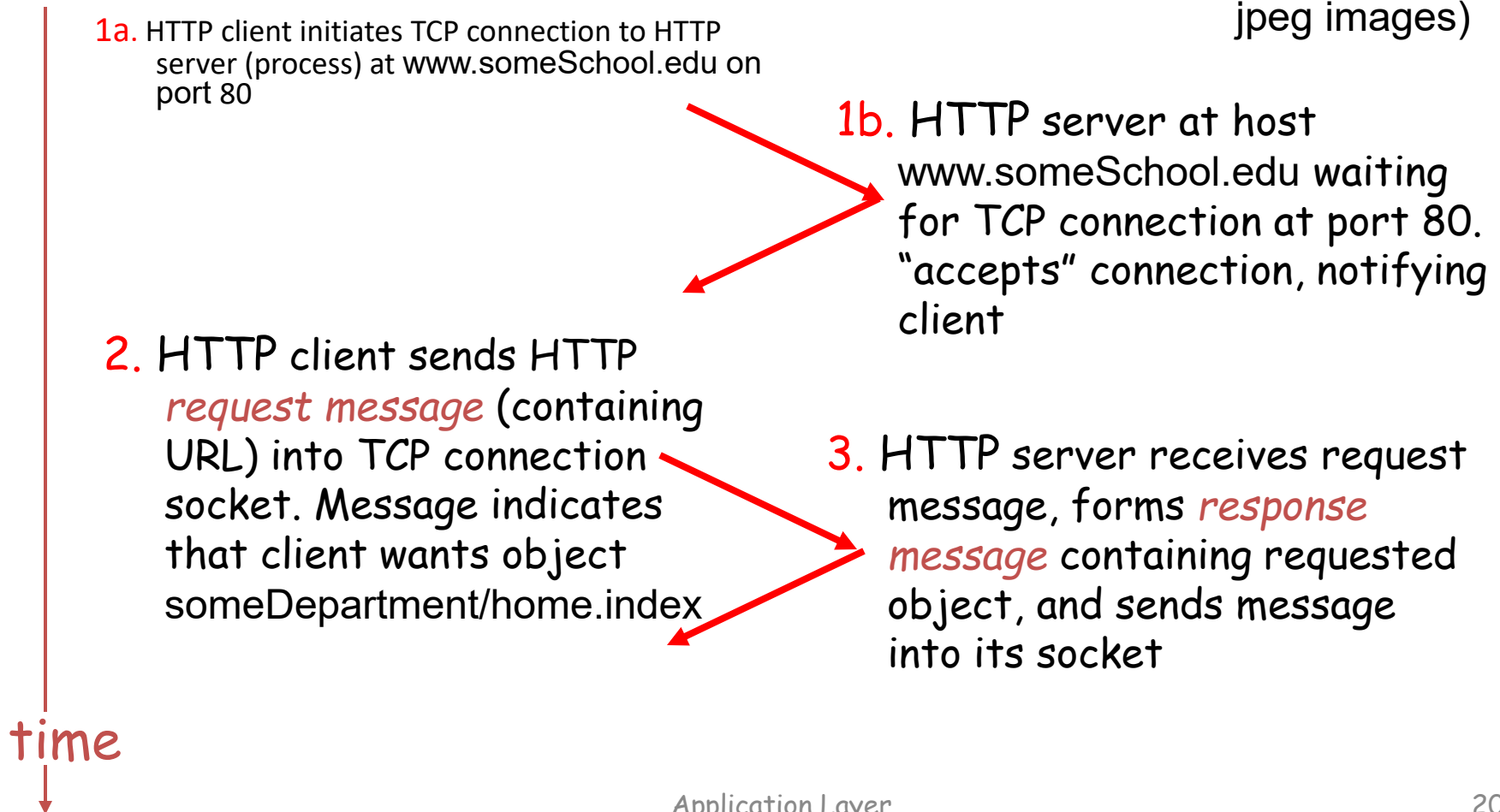
Persistent HTTP

- Multiple objects can be sent over single TCP connection between client and server.
- HTTP/1.1 uses persistent connections in default mode

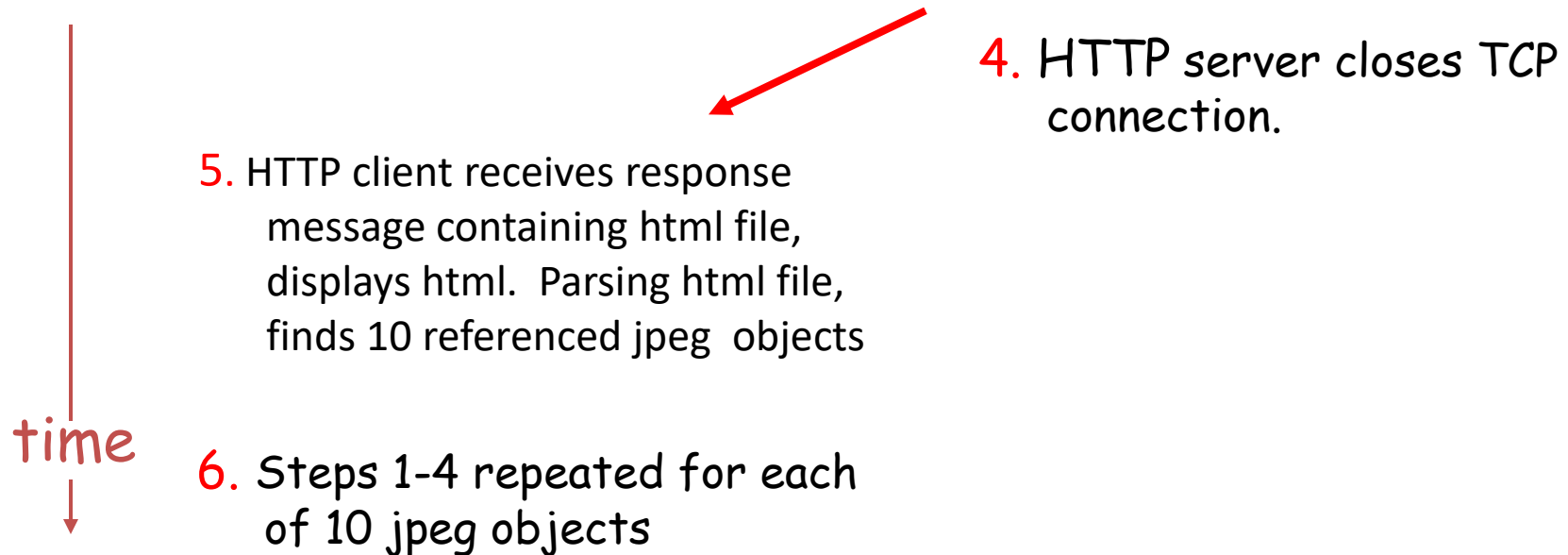
Nonpersistent HTTP [6]

Suppose user enters URL `www.someSchool.edu/someDepartment/home.index`

(contains text,
references to 10
jpeg images)



Nonpersistent HTTP (cont.) [6]



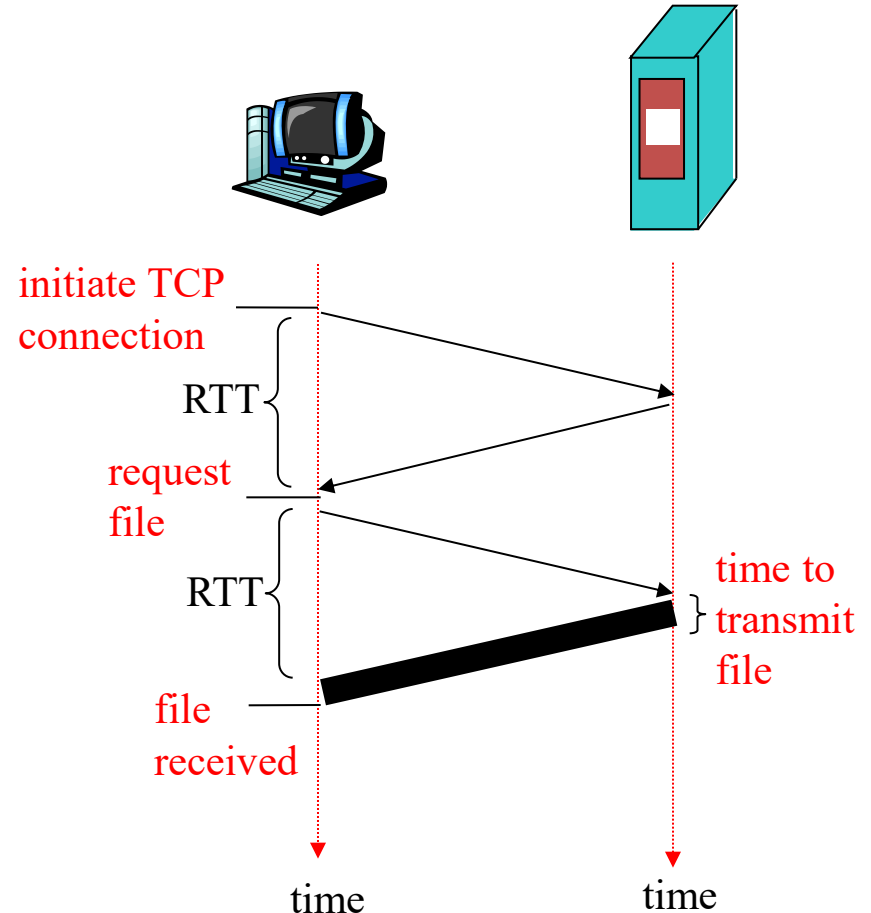
Non-Persistent HTTP: Response time [6]

Definition of RTT: time to send a small packet to travel from client to server and back.

Response time:

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- file transmission time

total = $2RTT + \text{transmit time}$



Persistent HTTP [6]

Nonpersistent HTTP issues:

- requires 2 RTTs per object
- OS overhead for *each* TCP connection
- browsers often open parallel TCP connections to fetch referenced objects

Persistent HTTP

- server leaves connection open after sending response
- subsequent HTTP messages between same client/server sent over open connection

Persistent **without** pipelining:

- client issues new request only when previous response has been received
- one RTT for each referenced object

Persistent **with** pipelining:

- default in HTTP/1.1
- client sends requests as soon as it encounters a referenced object
- as little as one RTT for all the referenced objects

HTTP request message [6]

- two types of HTTP messages: *request, response*
- HTTP request message:
 - ASCII (human-readable format)

request line
(GET, POST,
HEAD commands)

header
lines

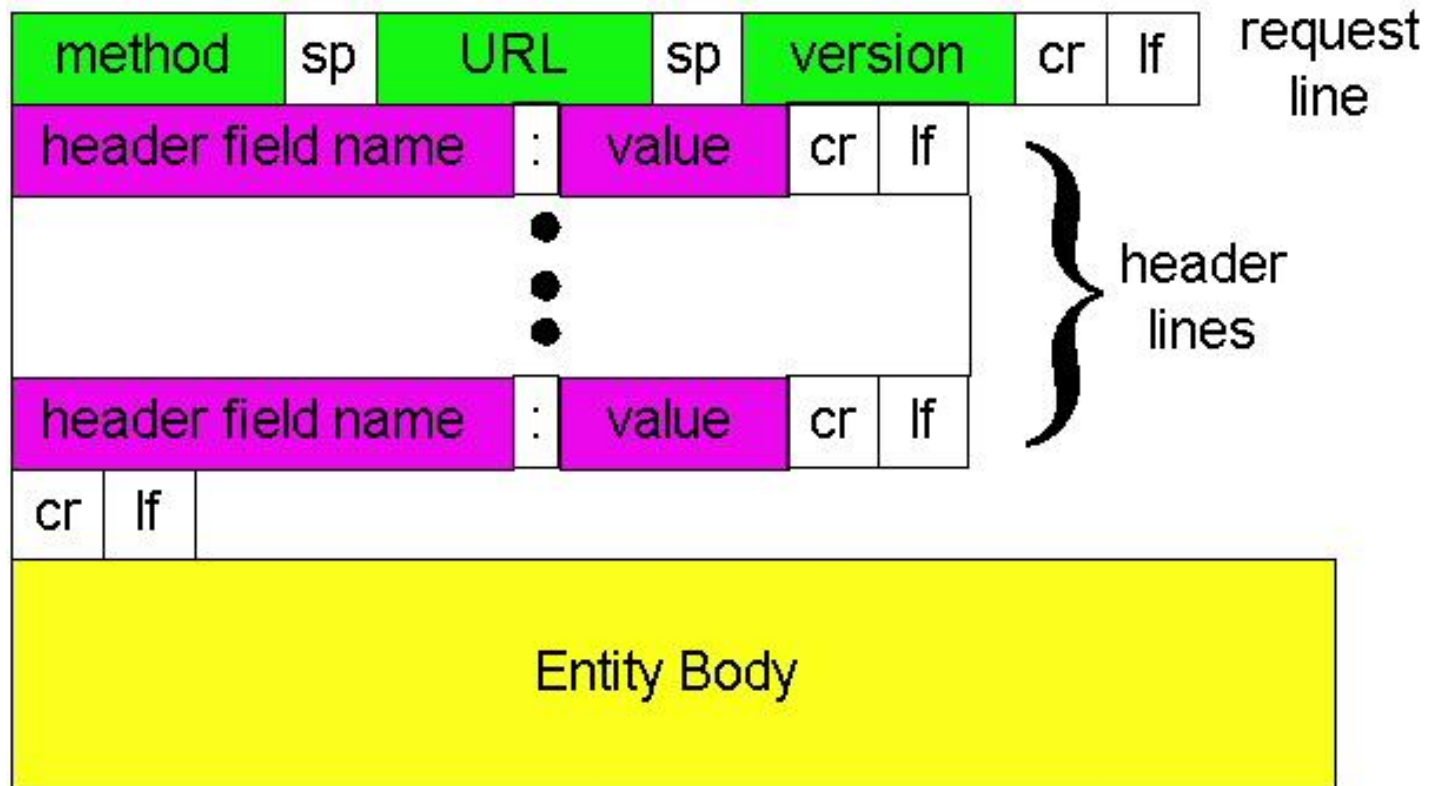
Carriage return,
line feed
indicates end
of message

GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
User-agent: Mozilla/4.0
Connection: close
Accept-language: fr

(extra carriage return, line feed)

The diagram illustrates the structure of an HTTP request message. It shows a sequence of lines: a request line, followed by header lines, and a final carriage return and line feed. Red arrows point from the descriptive text on the left to the corresponding parts of the message. The request line is 'GET /somedir/page.html HTTP/1.1'. The header lines are 'Host: www.someschool.edu', 'User-agent: Mozilla/4.0', 'Connection: close', and 'Accept-language: fr'. The carriage return and line feed are represented by a red arrow pointing to the end of the message.

HTTP request message: general format [6]



Uploading form input

Post method:

- Web page often includes form input
- Input is uploaded to server in entity body

URL method:

- Uses GET method
- Input is uploaded in URL field of request line:

`www.somesite.com/animalsearch?monkeys&banana`

Method types [6]

The first line of an HTTP request message is called the **request line**; the subsequent lines are called the **header lines**. **The request line has three fields: the method field, the URL field, and the HTTP version field.**

HTTP/1.0

- GET
- POST
- HEAD
 - asks server to leave requested object out of response

HTTP/1.1

- GET, POST, HEAD
- PUT
 - uploads file in entity body to path specified in URL field
- DELETE
 - deletes file specified in the URL field

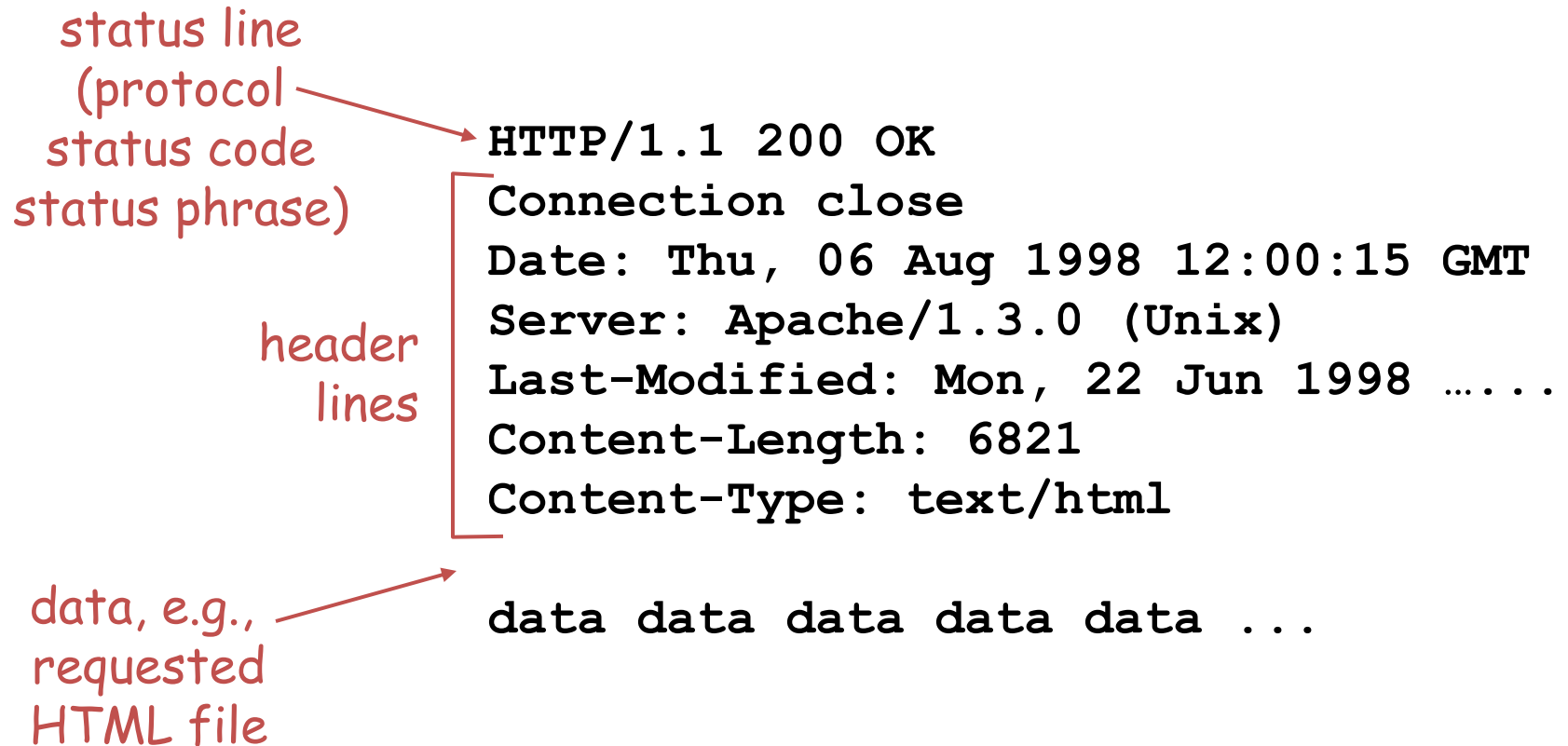
GET vs POST Method

- The GET method is used when the browser requests an object, with the requested object identified in the URL field.
- An HTTP client often uses the POST method when the user fills out a form—for example, when a user provides search words to a search engine.
- With a POST message, the user is still requesting a Web page from the server, but the specific contents of the Web page depend on what the user entered into the form fields.

HEAD, PUT, and DELETE

- The HEAD method is similar to the GET method. When a server receives a request with the HEAD method, it responds with an HTTP message but it leaves out the requested object. Application developers often use the HEAD method for debugging.
- The PUT method is often used in conjunction with Web publishing tools. It allows a user to upload an object to a specific path (directory) on a specific Web server.
- The PUT method is also used by applications that need to upload objects to Web servers.
- The DELETE method allows a user, or an application, to delete an object on a Web server.

HTTP response message [6]



HTTP response status codes

In first line in server->client response message.

A few sample codes:

200 OK

- request succeeded, requested object later in this message

301 Moved Permanently

- requested object moved, new location specified later in this message (Location:)

400 Bad Request

- request message not understood by server

404 Not Found

- requested document not found on this server

505 HTTP Version Not Supported

User-server state: cookies

Many major Web sites use cookies

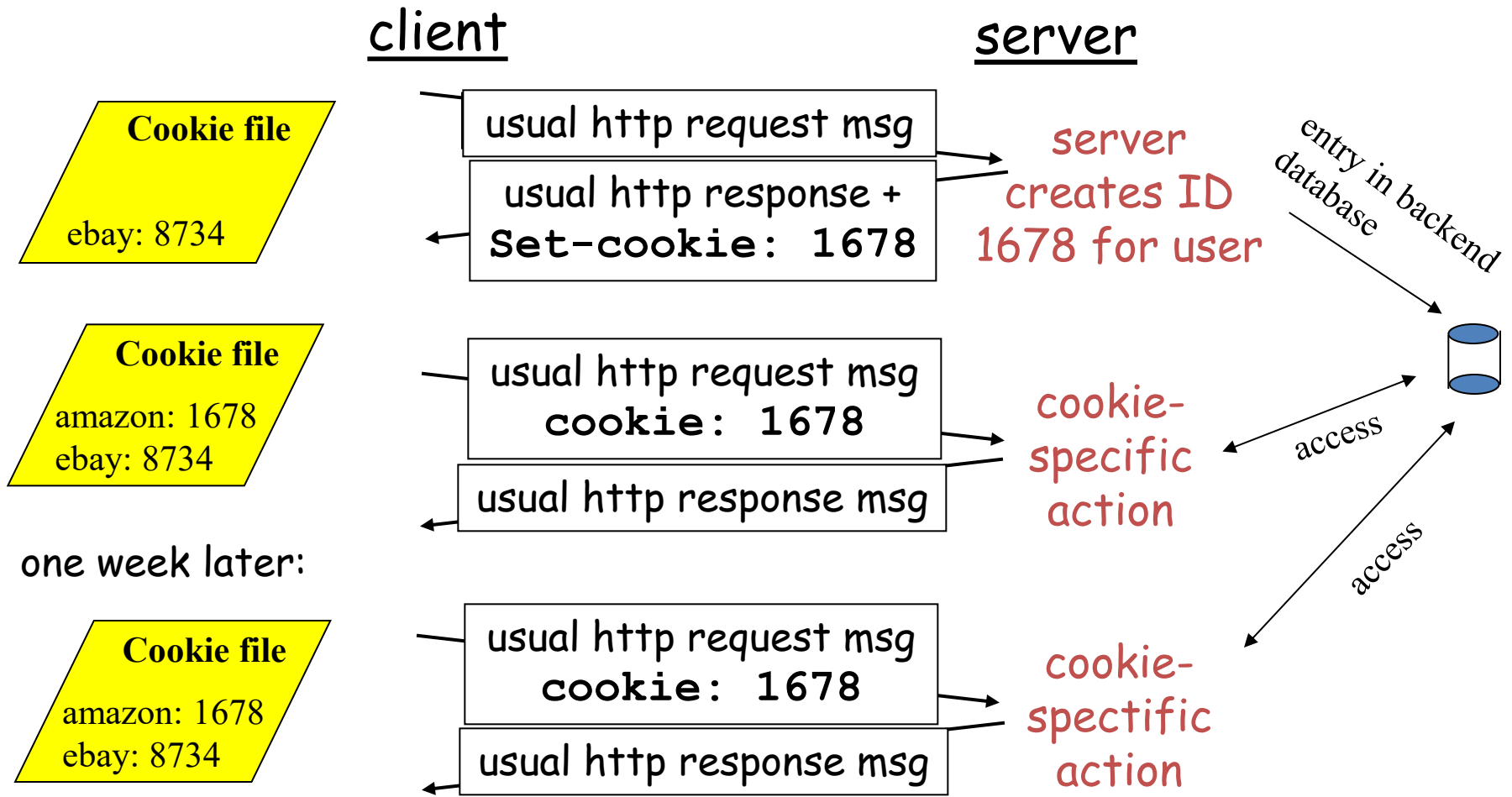
Four components:

- 1) cookie header line of HTTP *response* message
- 2) cookie header line in HTTP *request* message
- 3) cookie file kept on user's host, managed by user's browser
- 4) back-end database at Web site

Example:

- Susan access Internet always from same PC
- She visits a specific e-commerce site for first time
- When initial HTTP requests arrives at site, site creates a unique ID and creates an entry in backend database for ID

Cookies: keeping “state” (cont.) [6]



Cookies (continued)

What cookies can bring:

- authorization
- shopping carts
- recommendations
- user session state (Web e-mail)

Cookies and privacy: aside

- ☐ cookies permit sites to learn a lot about you
- ☐ you may supply name and e-mail to sites

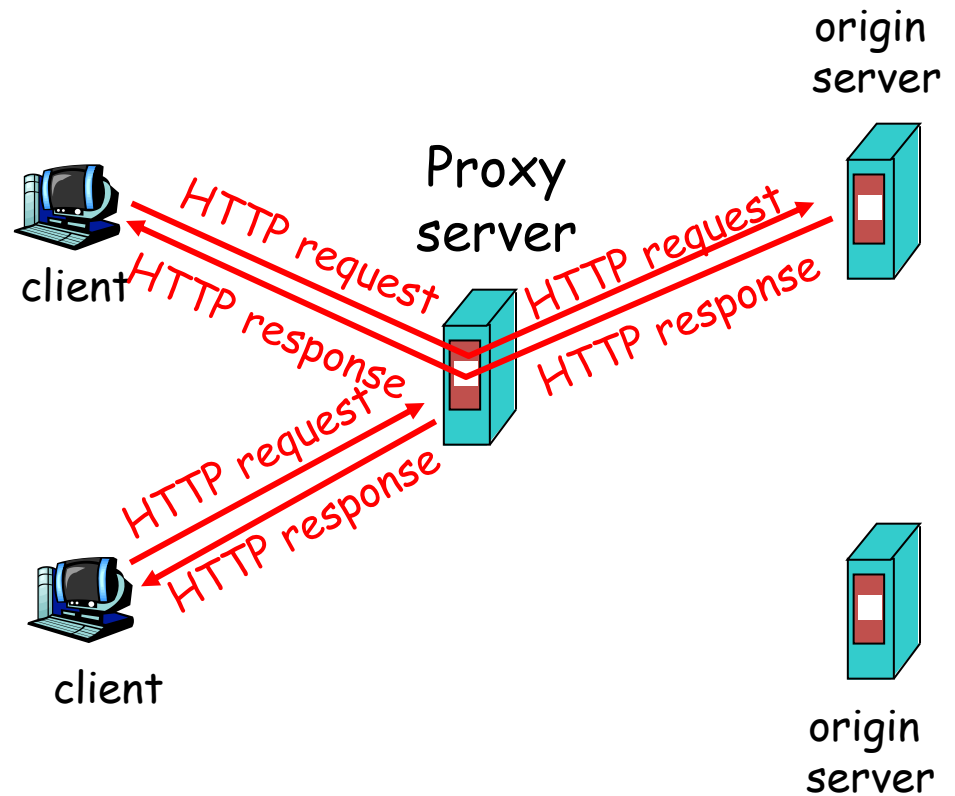
How to keep "state":

- ☐ Protocol endpoints: maintain state at sender/receiver over multiple transactions
- ☐ cookies: http messages carry state

Web caches (proxy server) [6]

Goal: satisfy client request without involving origin server

- user sets browser: Web accesses via cache
- browser sends all HTTP requests to cache
 - object in cache: cache returns object
 - else cache requests object from origin server, then returns object to client



More about Web caching

- Cache acts as both client and server
- Typically cache is installed by ISP (university, company, residential ISP)

Why Web caching?

- Reduce response time for client request.
- Reduce traffic on an institution's access link.
- Internet dense with caches: enables “poor” content providers to effectively deliver content

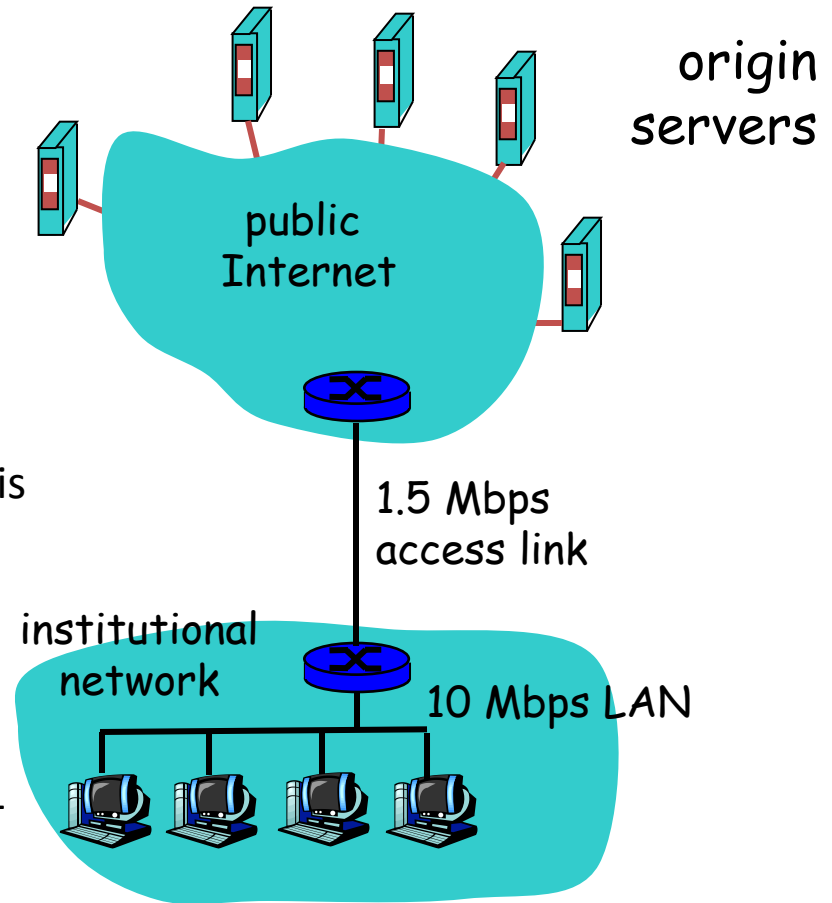
Caching example [6]

Assumptions

- average object size = 100,000 bits
- avg. request rate from institution's browsers to origin servers = 15/sec
- delay from access link router to any origin server and back to router = 2 sec

Consequences

- Utilization or traffic intensity, $I = \Delta\beta$, where β is an average arrival rate, Δ is an average transmission time which is L/R .
- utilization on LAN = 15%
- utilization on access link = 100%
- total delay = Internet delay + access delay + LAN delay
- Average delay = $\Delta / (1 - I)$,
= 2 sec + minutes + milliseconds
= 2 sec + 60 sec + $\{(0.01 / (1 - 0.15)) \text{ sec} \rightarrow 11.7 \text{ ms}\}$



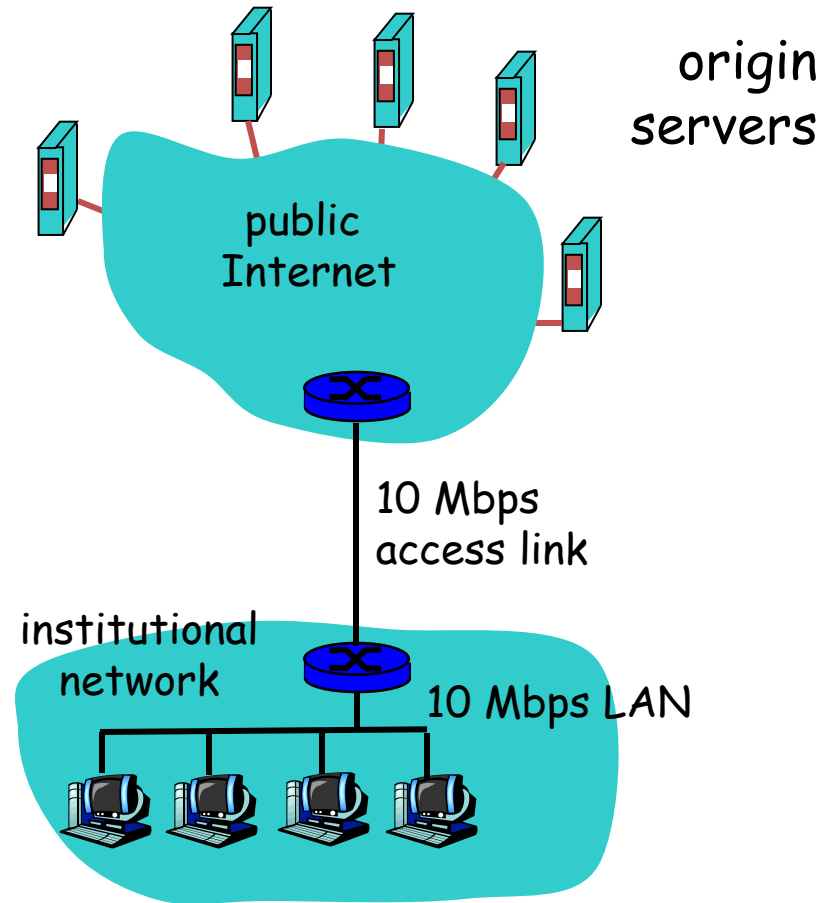
Caching example (cont) [6]

Possible solution

- increase bandwidth of access link to, say, 10 Mbps

Consequences

- utilization on LAN = 15%
- utilization on access link = 15%
- Total delay = Internet delay + access delay + LAN delay
= 2 sec + msec + msec
- often a costly upgrade



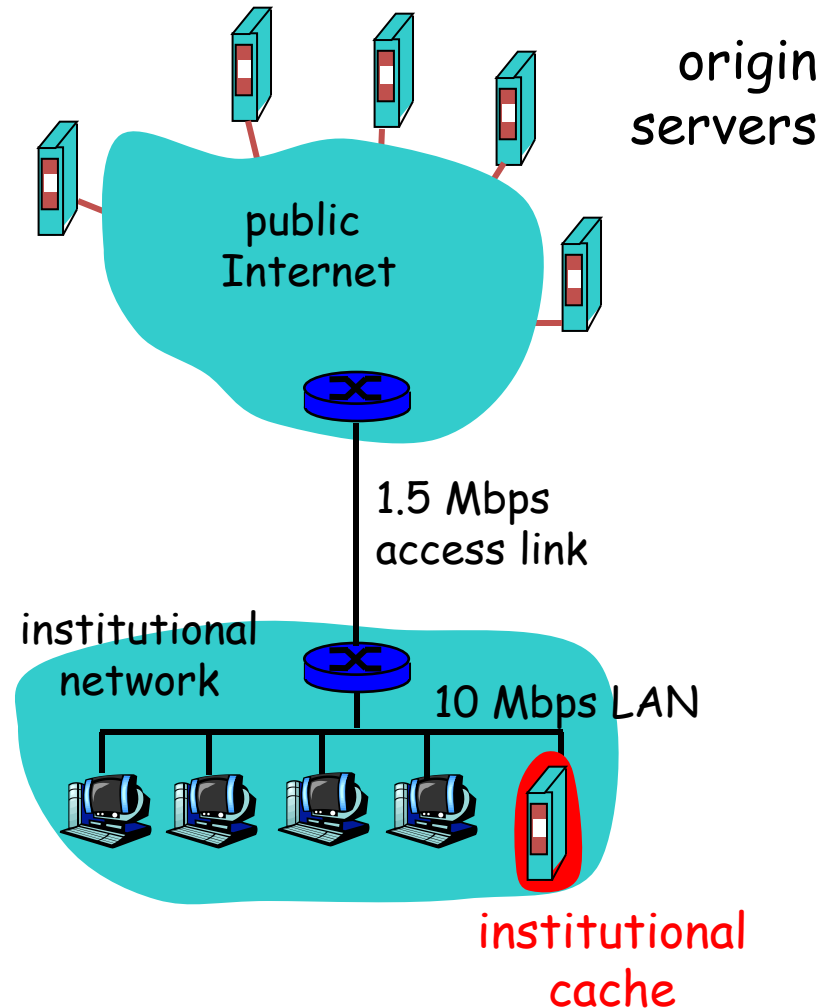
Caching example (cont) [6]

Install cache

- suppose hit rate is .4

Consequence

- 40% requests will be satisfied almost immediately
- 60% requests satisfied by origin server
- utilization of access link reduced to 60%, resulting in negligible delays (say 10 msec)
- total avg delay = Internet delay + access delay + LAN delay = $0.6 \cdot (2.01) \text{ seconds} + 0.4 \cdot \text{msec}$
= 1.21 seconds



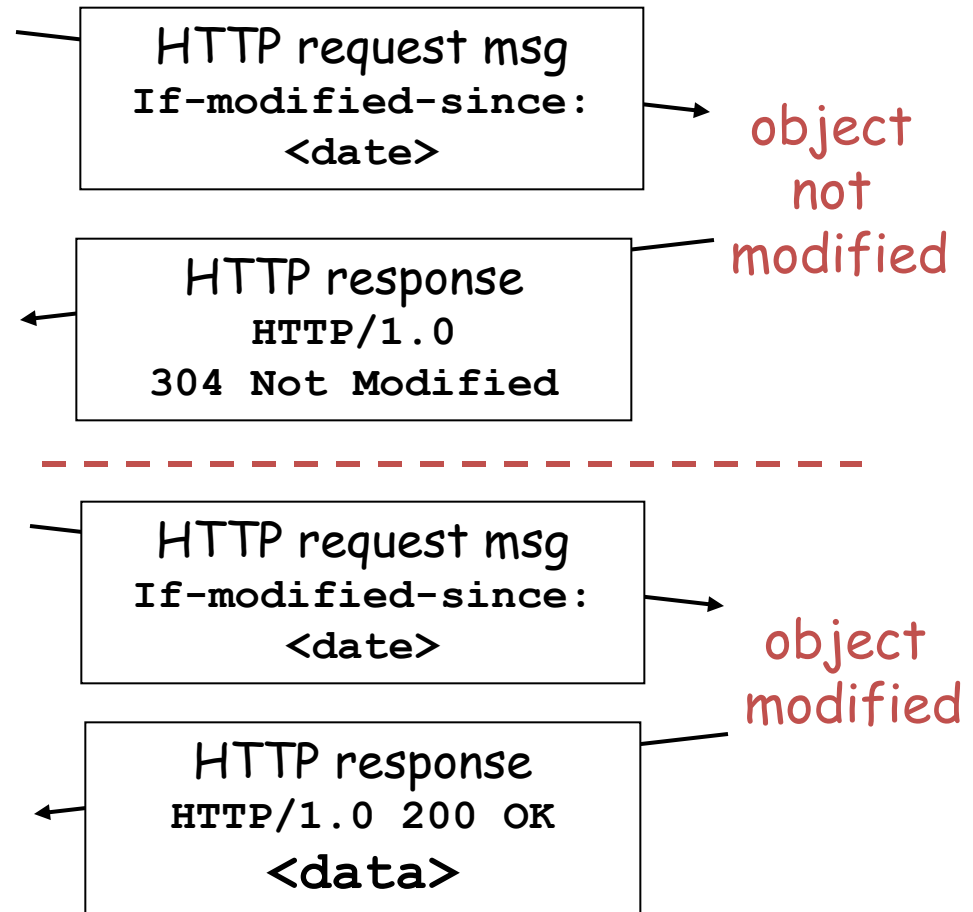
Conditional GET [6]

- **Goal:** don't send object if cache has up-to-date cached version
- cache: specify date of cached copy in HTTP request
If-modified-since:
<date>
- server: response contains no object if cached copy is up-to-date:

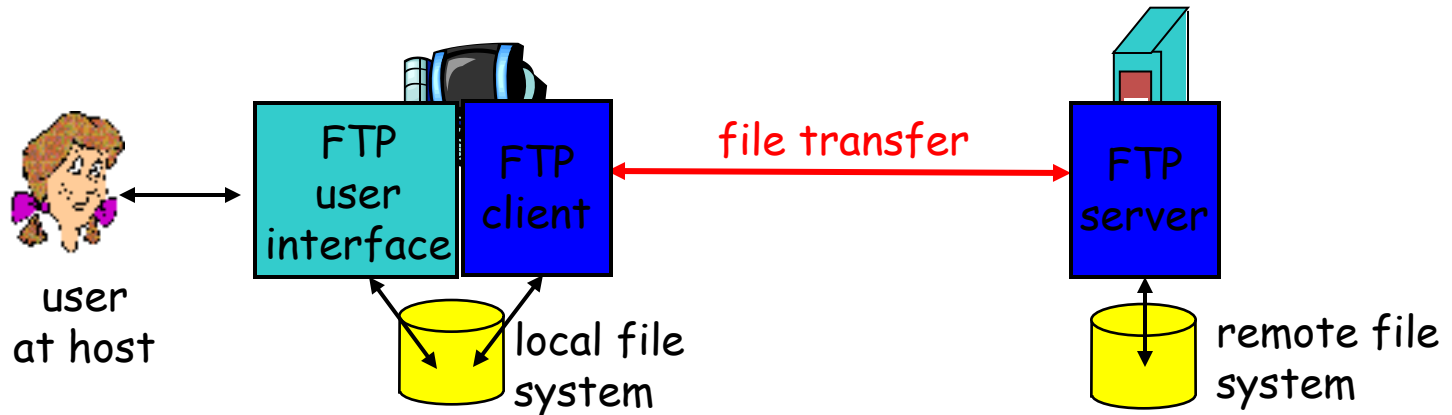
**HTTP/1.0 304 Not
Modified**

cache

server



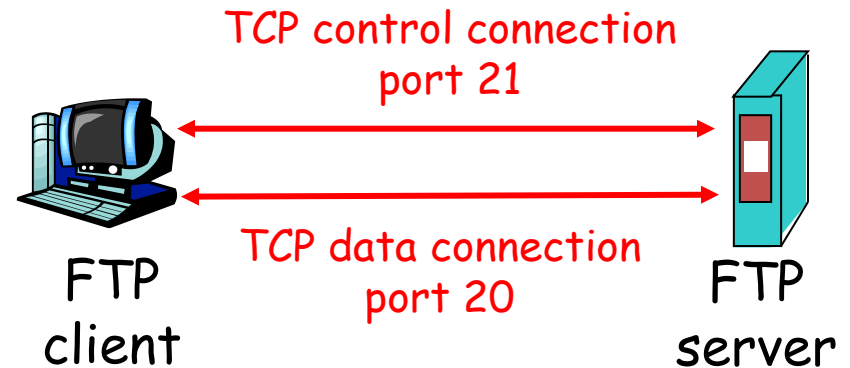
FTP: the file transfer protocol [6]



- transfer file to/from remote host
- client/server model
 - *client*: side that initiates transfer (either to/from remote)
 - *server*: remote host
- ftp: RFC 959
- ftp server: port 21

FTP: separate control, data connections [6]

- FTP client contacts FTP server at port 21, specifying TCP as transport protocol
- Client obtains authorization over control connection
- Client browses remote directory by sending commands over control connection.
- When server receives file transfer command, server opens 2nd TCP connection (for file) to client
- After transferring one file, server closes data connection.



- ❑ Server opens another TCP data connection to transfer another file.
- ❑ Control connection: "out of band"
- ❑ FTP server maintains "state": current directory, earlier authentication

FTP commands, responses

Sample commands:

- sent as ASCII text over control channel
- **USER** *username*
- **PASS** *password*
- **LIST** return list of file in current directory
- **RETR filename** retrieves (gets) file
- **STOR filename** stores (puts) file onto remote host

Sample return codes

- status code and phrase (as in HTTP)
- 331 Username OK, password required
- 125 data connection already open; transfer starting
- 425 Can't open data connection
- 452 Error writing file

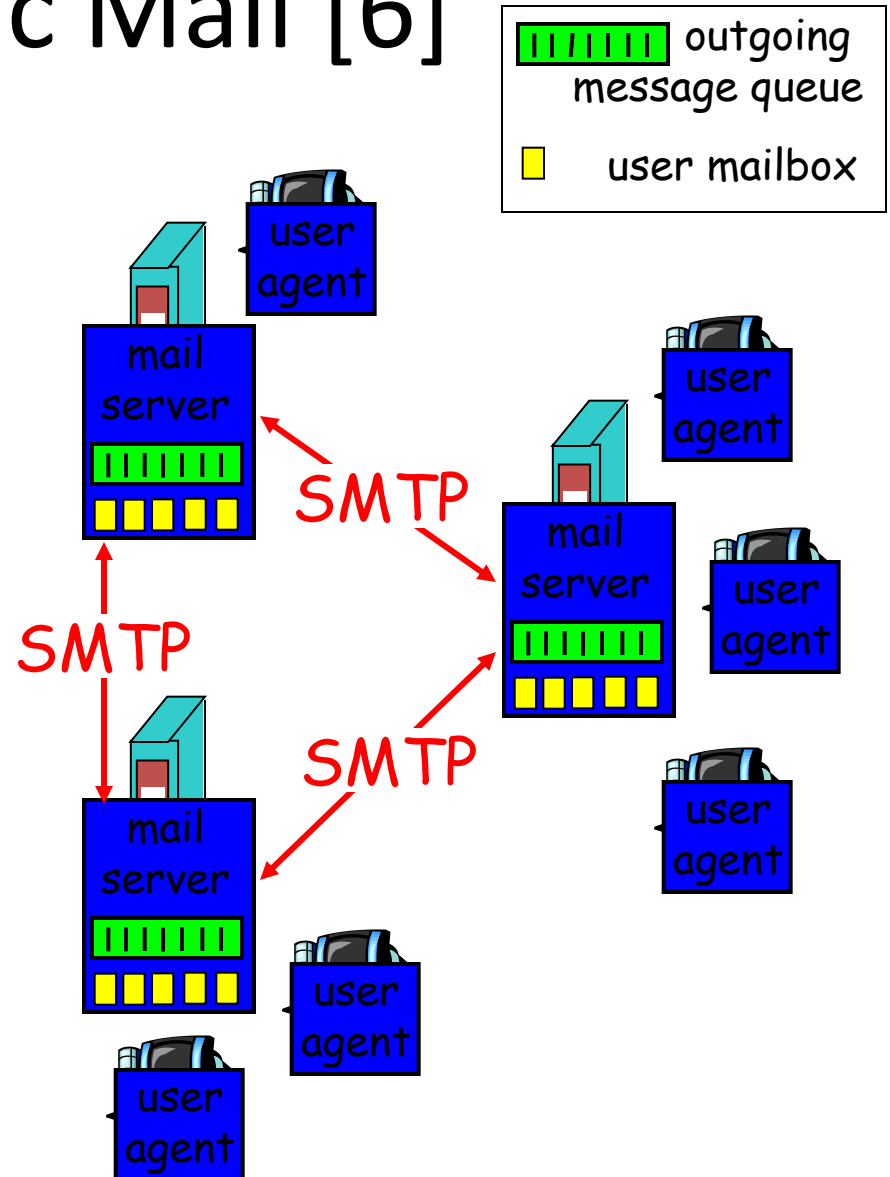
Electronic Mail [6]

Three major components:

- user agents
- mail servers
- simple mail transfer protocol: SMTP

User Agent

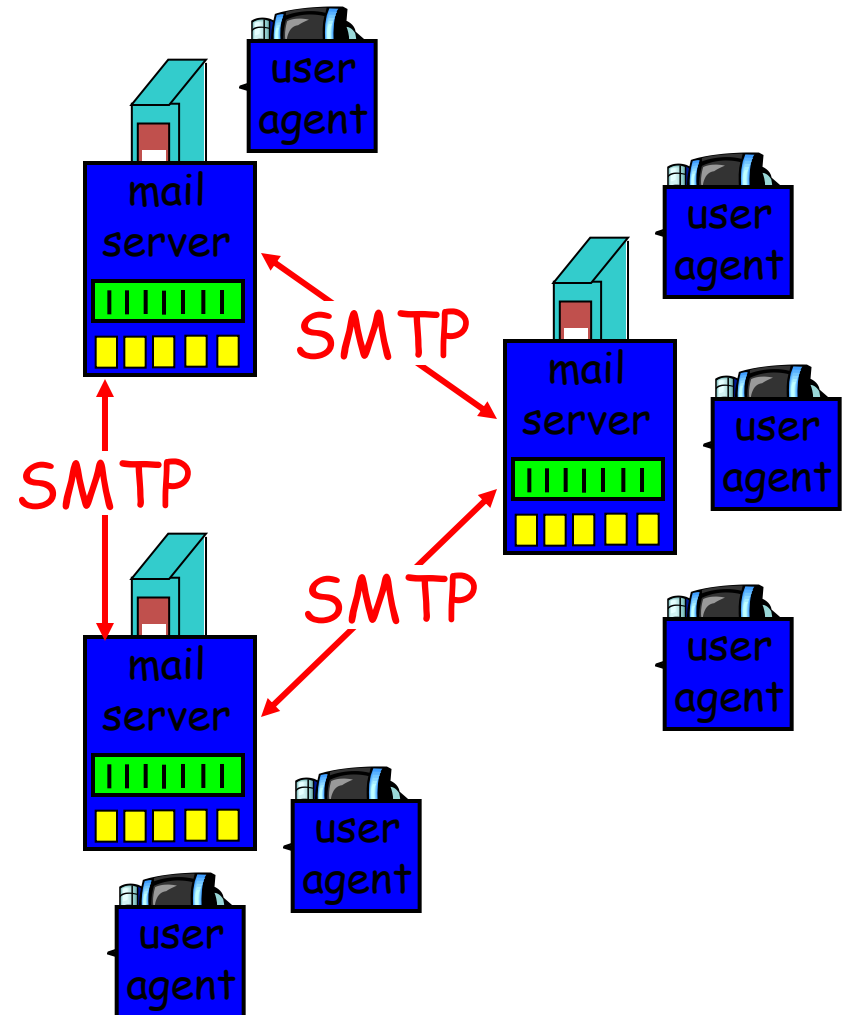
- a.k.a. “mail reader”
- composing, editing, reading mail messages
- e.g., Eudora, Outlook, elm, Netscape Messenger
- outgoing, incoming messages stored on server



Electronic Mail: mail servers

Mail Servers

- **mailbox** contains incoming messages for user
- **message queue** of outgoing (to be sent) mail messages
- **SMTP protocol** between mail servers to send email messages
 - client: sending mail server
 - “server”: receiving mail server

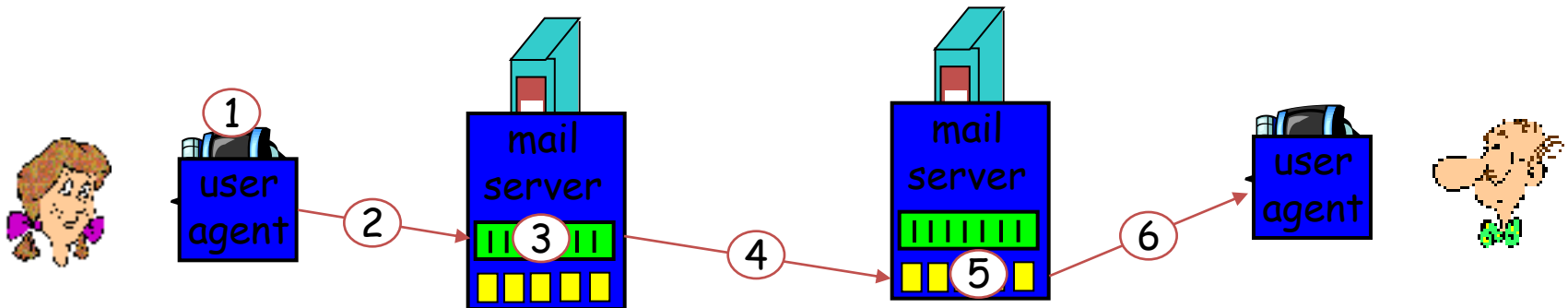


Electronic Mail: SMTP [RFC 2821]

- uses TCP to reliably transfer email message from client to server, port 25
- direct transfer: sending server to receiving server
- three phases of transfer
 - handshaking (greeting)
 - transfer of messages
 - closure
- command/response interaction
 - **commands**: ASCII text
 - **response**: status code and phrase
- messages must be in 7-bit ASCII

Scenario: Alice sends message to Bob

- 1) Alice uses UA to compose message and "to" `bob@someschool.edu`
- 2) Alice's UA sends message to her mail server; message placed in message queue
- 3) Client side of SMTP opens TCP connection with Bob's mail server
- 4) SMTP client sends Alice's message over the TCP connection
- 5) Bob's mail server places the message in Bob's mailbox
- 6) Bob invokes his user agent to read message



SMTP: final words [6]

- SMTP uses persistent connections
- SMTP requires message (header & body) to be in 7-bit ASCII
- SMTP server uses `CRLF` . `CRLF` to determine end of message
- **Pull Protocol:** Someone loads information on a Web server and users use HTTP to pull the information from the server at their convenience (receive file).
- **Push Protocol:** The sending mail server pushes the file to the receiving mail server (send file)

Comparison with HTTP:

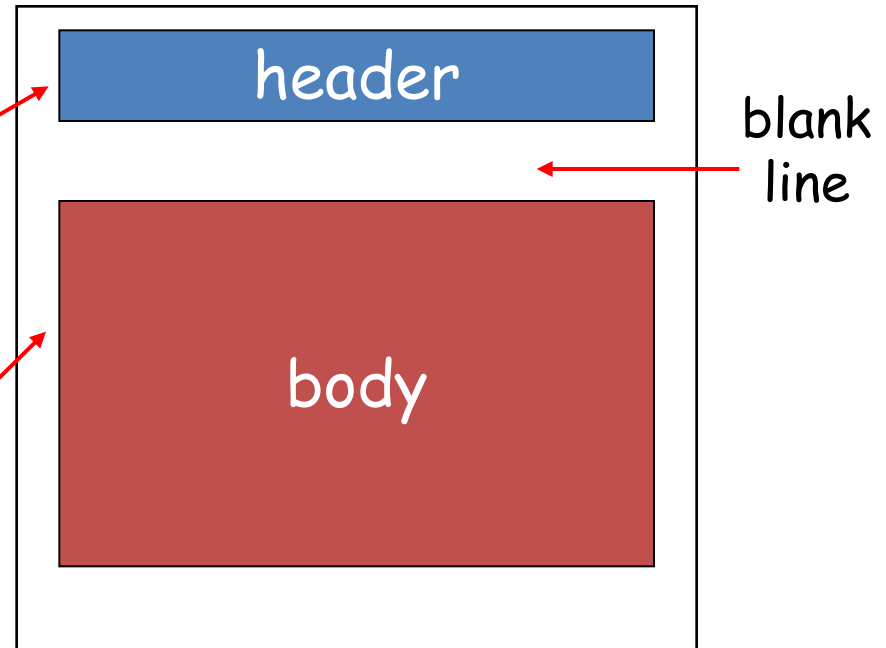
- HTTP: pull
- SMTP: push
- both have ASCII command/response interaction, status codes
- HTTP: each object encapsulated in its own response message.
- SMTP: multiple objects sent in multipart message.

Mail message format [6]

SMTP: protocol for exchanging email
msgs

RFC 822: standard for text message
format:

- header lines, e.g.,
 - To:
 - From:
 - Subject:*different from SMTP commands!*
- body
 - the “message”, ASCII characters only



Message format: multimedia extensions [6]

- Multipurpose Internet Mail Extensions (MIME): multimedia mail extension, RFC 2045, 2056
- additional lines in msg header declare MIME content type

MIME version

method used
to encode data

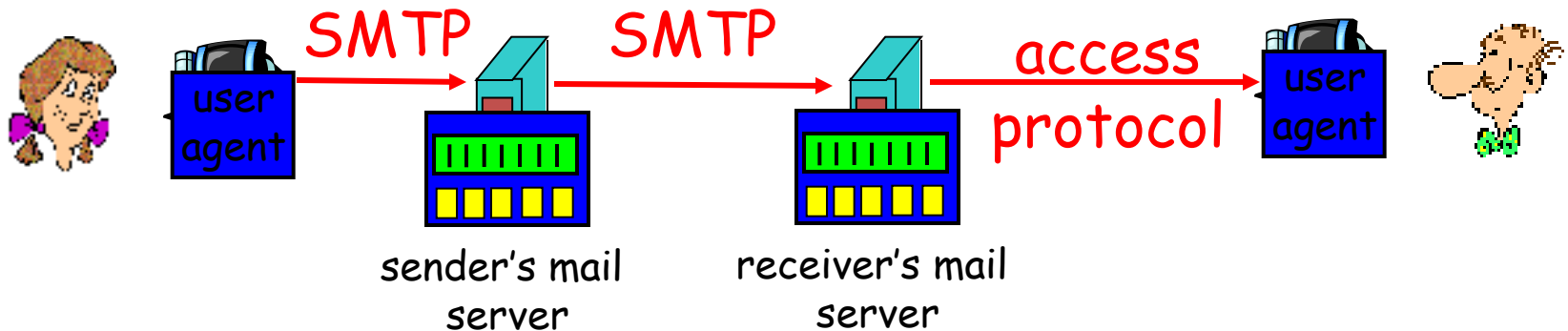
multimedia data
type, subtype,
parameter declaration

encoded data

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Picture of yummy crepe.
MIME-Version: 1.0
Content-Transfer-Encoding: base64
Content-Type: image/jpeg

base64 encoded data .....
.....
.....base64 encoded data
```

Mail access protocols [6]



- SMTP: delivery/storage to receiver's server
- Mail access protocol: retrieval from server
 - POP: Post Office Protocol [RFC 1939]
 - authorization (agent <-->server) and download
 - IMAP: Internet Mail Access Protocol [RFC 1730]
 - more features (more complex)
 - manipulation of stored msgs on server
 - HTTP: Hotmail , Yahoo! Mail, etc.

POP3 protocol [6]

authorization phase

- client commands:
 - **user**: declare username
 - **pass**: password
- server responses
 - **+OK**
 - **-ERR**



```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on
```

transaction phase, client:

- **list**: list message numbers
- **retr**: retrieve message by number
- **dele**: delete
- **quit**



```
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

POP3 (more) and IMAP

More about POP3

- Previous example uses “download and delete” mode.
- Bob cannot re-read e-mail if he changes client
- “Download-and-keep”: copies of messages on different clients
- POP3 is stateless across sessions

IMAP

- Keep all messages in one place: the server
- Allows user to organize messages in folders
- IMAP keeps user state across sessions:
 - names of folders and mappings between message IDs and folder name

DNS: Domain Name System

People: many identifiers:

- SSN, name, passport #

Internet hosts, routers:

- IP address (32 bit) - used for addressing datagrams
- “name”, e.g., ww.yahoo.com - used by humans

Q: map between IP addresses and name ?

Domain Name System:

- *distributed database* implemented in hierarchy of many *name servers*
- *application-layer protocol* host, routers, name servers to communicate to *resolve* names (address/name translation)
 - note: core Internet function, implemented as application-layer protocol
 - complexity at network’s “edge”

DNS

DNS services

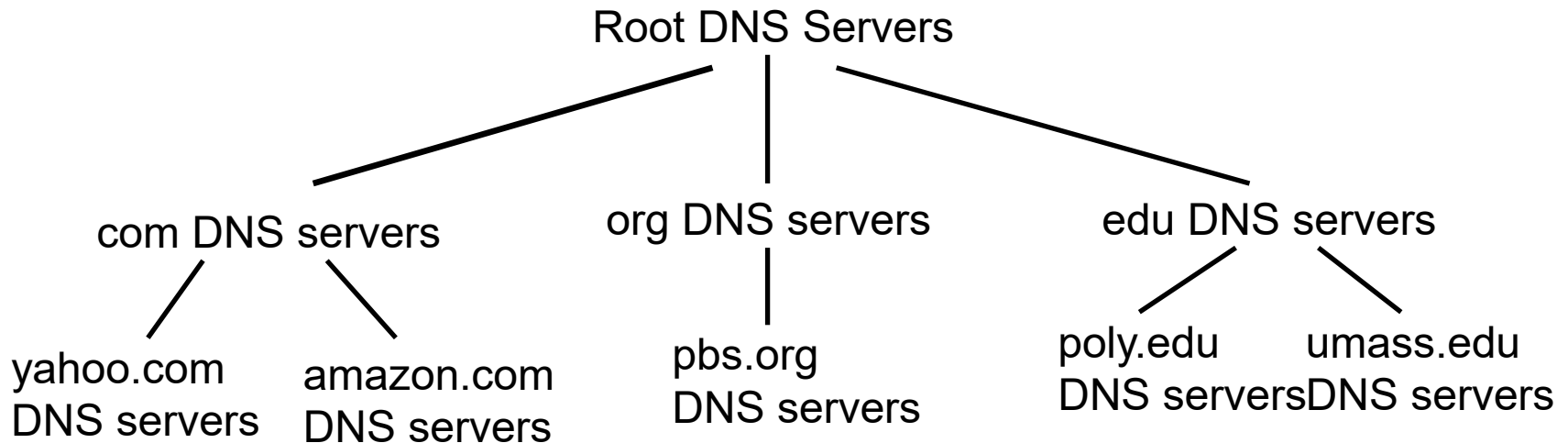
- Hostname to IP address translation
- Host aliasing
 - Canonical and alias names
- Mail server aliasing
- All DNS servers use port 53 with UDP datagram
- Load distribution
 - Replicated Web servers: set of IP addresses for one canonical name

Why not centralize DNS?

- single point of failure
- traffic volume
- distant centralized database
- maintenance

doesn't *scale!*

Distributed, Hierarchical Database [6]

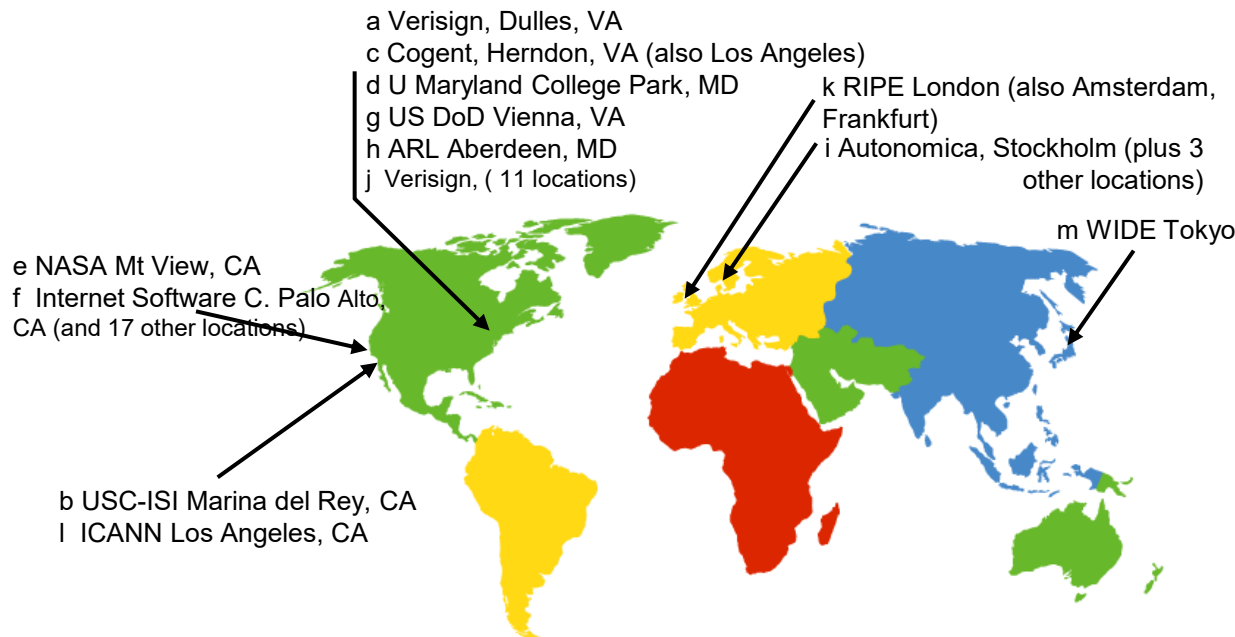


Client wants IP for www.amazon.com; 1st approx:

- Client queries a root server to find com DNS server
- Client queries com DNS server to get amazon.com DNS server
- Client queries amazon.com DNS server to get IP address for www.amazon.com

DNS: Root name servers [6]

- contacted by local name server that can not resolve name
- root name server:
 - contacts authoritative name server if name mapping not known
 - gets mapping
 - returns mapping to local name server



13 root name
servers worldwide

TLD and Authoritative Servers

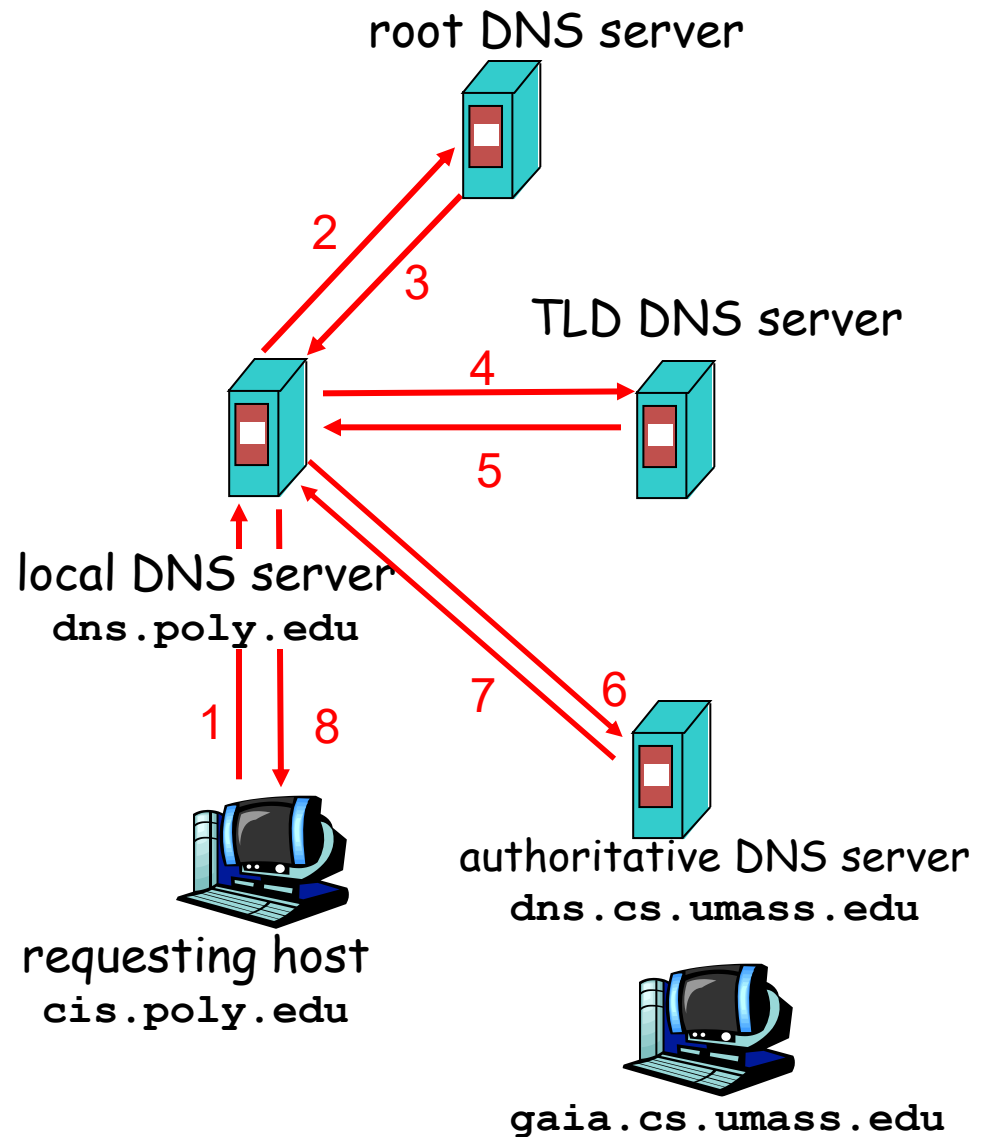
- **Top-level domain (TLD) servers:** responsible for com, org, net, edu, etc, and all top-level country domains uk, fr, ca, jp.
 - Network solutions maintains servers for com TLD
 - Educause for edu TLD
- **Authoritative DNS servers:** organization's DNS servers, providing authoritative hostname to IP mappings for organization's servers (e.g., Web and mail).
 - Can be maintained by organization or service provider

Local Name Server

- Does not strictly belong to hierarchy
- Each ISP (residential ISP, company, university) has one.
 - Also called “default name server”
- When a host makes a DNS query, query is sent to its local DNS server
 - Acts as a proxy, forwards query into hierarchy.

Example [6]

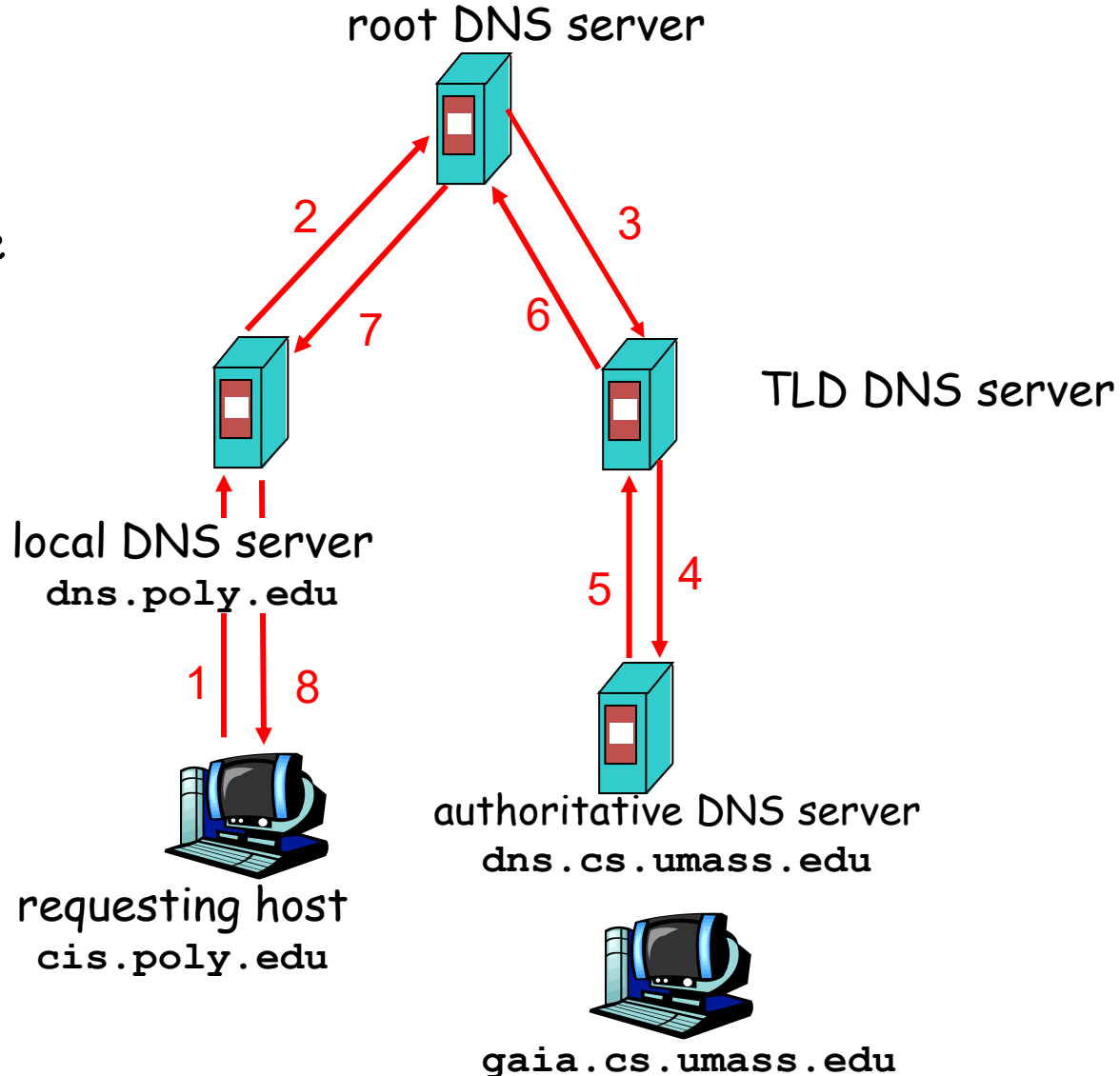
- Host at cis.poly.edu wants IP address for gaia.cs.umass.edu
- iterated query:
- contacted server replies with name of server to contact
- “I don’t know this name, but ask this server”



Recursive queries [6]

recursive query:

- ❑ puts burden of name resolution on contacted name server
- ❑ heavy load?



DNS: caching and updating records

- once (any) name server learns mapping, it *caches* mapping
 - cache entries timeout (disappear) after some time
 - TLD servers typically cached in local name servers
 - Thus, root name servers not often visited
- update/notify mechanisms under design by Internet Engineering Task Force (IETF)
 - RFC 2136
 - <http://www.ietf.org/html.charters/dnsind-charter.html>

DNS records [6]

DNS: distributed db storing resource records (RR)

RR format: (name, value, type, ttl)

□ Type=A

- ❖ **name** is hostname
- ❖ **value** is IP address

• Type=NS

- **name** is domain (e.g. foo.com)
- **value** is hostname of authoritative name server/TLD server for this domain

□ Type=CNAME

- ❖ **name** is alias name for some "canonical" (the real) name
www.ibm.com is really
servereast.backup2.ibm.com
- ❖ **value** is canonical name

□ Type=MX

- ❖ **value** is name of mailserver associated with name
- ❖ **Name** is domain name

DNS protocol, messages [6]

DNS protocol : *query* and *reply* messages, both with same *message format*

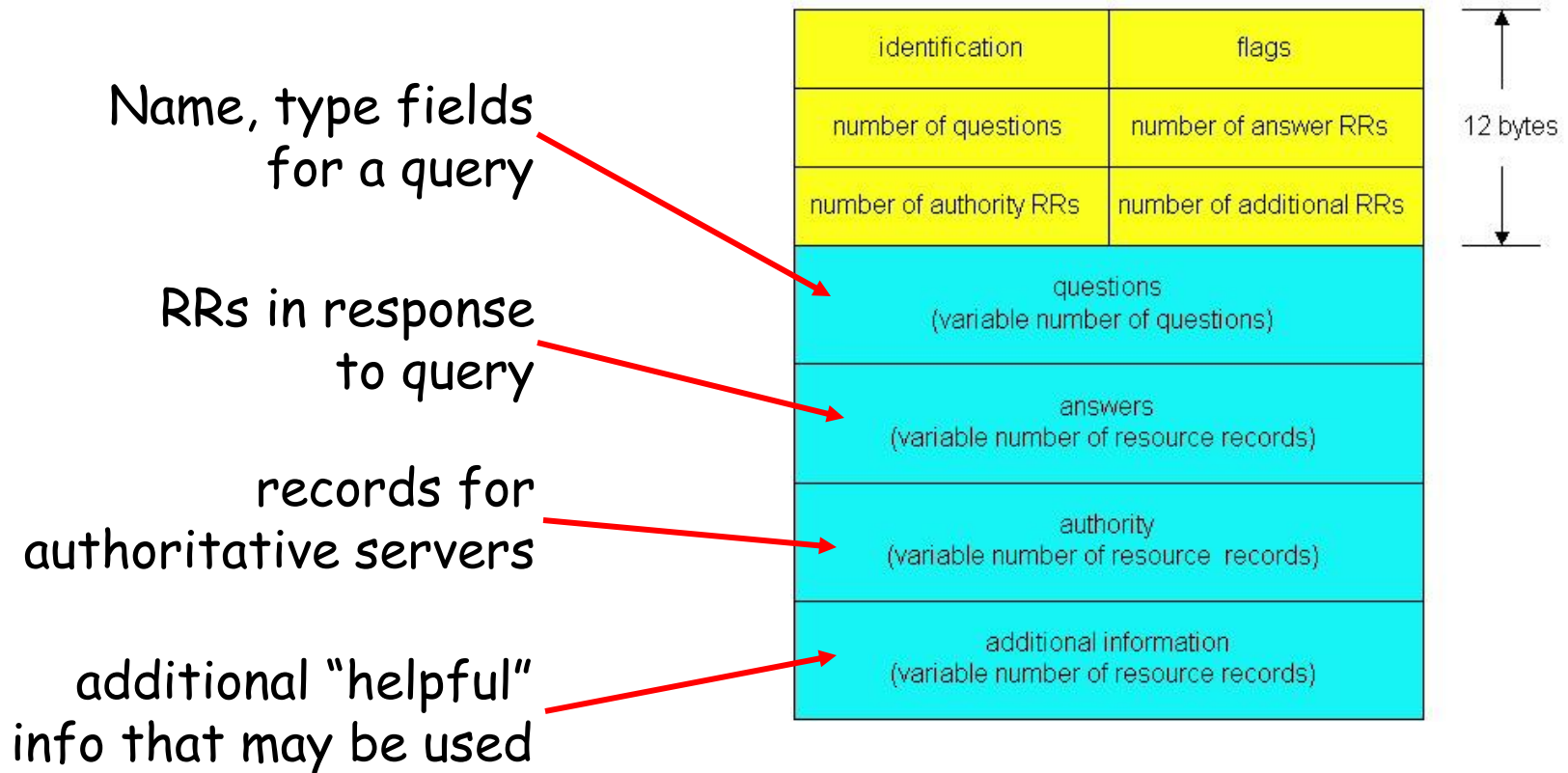
msg header

- ❑ **identification**: 16 bit, for query, reply to query, both messages have the same format
- ❑ **flags**: (1-bit each)
 - ❖ query or reply
 - ❖ recursion desired
 - ❖ recursion available
 - ❖ reply is authoritative

identification	flags
number of questions	number of answer RRs
number of authority RRs	number of additional RRs
questions (variable number of questions)	
answers (variable number of resource records)	
authority (variable number of resource records)	
additional information (variable number of resource records)	

↑
12 bytes
↓

DNS protocol, messages [6]



Inserting records into DNS [6]

- Example: just created startup “Network Utopia”
- Register name networkutopia.com at a registrar (e.g., Network Solutions)
 - Need to provide registrar with names and IP addresses of your authoritative name server (primary and secondary)
 - Registrar inserts two RRs into the com TLD server:
 - (networkutopia.com, dns1.networkutopia.com, NS)
 - (dns1.networkutopia.com, 212.212.212.1, A)
- Put in authoritative server Type A record for web server: www.networkutopia.com and Type MX record for mail server: networkutopia.com
 - Registrar inserts RRs into the authoritative name server:
 - (networkutopia.com, mail.networkutopia.com, MX)
 - (mail.networkutopia.com, 212.212.71.4, A)
 - (networkutopia.com, www.networkutopia.com, CNAME)
 - (www.networkutopia.com, 212.212.71.5, A)
- How do people get the IP address of your Web site?

References

1. William Stallings, "Data and Computer Communications", Seventh Edition, PHI 2004.
2. Andrew S. Tanenbaum, "Computer Networks" 4th Edition PHI
3. B. A. Fourozan, "TCP/IP Protocol Suite", 3rd Edition, Singapore, McGrawHill, 2004.
4. L. L. Peterson and B. S. Davie, Computer Networks-A System Approach, Elsevier.
5. B. A. Fourozan, "Data Communications and Networking", 4th Edition, Singapore, McGrawHill, 2004.
6. James F. Kurose, Keith W. Ross, "Computer Networking: A Top-Down Approach Featuring the Internet", 3rd Edition , Pearson Education 2009.
7. https://gaia.cs.umass.edu/kurose_ross/ppt.htm
8. PPT available for the respective books

Thank You